# Tema Finale ISS 2026 – Sprint 1

Edoardo Buttazzi

## 1 Introduction

### 1.1 Risultati Sprint 0

Lo Sprint 0 ha individuato le macro-parti del sistema, distinto i componenti forniti da quelli da sviluppare e proposto `RobotSmart26` come supporto alla navigazione. Ha inoltre introdotto un primo modello QAK di `cargoservice` e `cargorobot`, descrivendo IOPort, sonar e LED attraverso le loro responsabilità funzionali.

Il modello riportato nello Sprint 0 costituisce un punto di partenza: esplicita le tre risposte alla richiesta di carico, mentre il movimento del robot è rappresentato da una risposta immediata e la conferma temporale del sonar è soltanto indicata. Il completamento di queste funzionalità rientra quindi nel lavoro dello Sprint 1.

### 1.2 Goal dello Sprint 1

L'obiettivo è ottenere un **prototipo eseguibile del ciclo di carico**, dalla richiesta del cliente al deposito del container nello slot riservato. Il ciclo comprende l'accettazione, la prenotazione, l'attesa del container, il trasferimento a `slot5`, l'attesa del completamento della marcatura, il trasferimento allo slot finale e il successivo rientro del robot in HOME.

In coerenza con lo Sprint 0, il prototipo dovrà:

1. gestire le richieste secondo la disponibilità della stiva, della IOPort e del servizio.
2. coordinare `cargoservice` e `cargorobot`, integrando la navigazione virtuale tramite `RobotSmart26`.
3. simulare pulsante, display e misure del sonar, mantenendo il vincolo temporale per il riconoscimento del container.
4. rappresentare gli slot di deposito e l'area di marcatura, distinguendo prenotazione e occupazione effettiva.

L'attuazione fisica del lampeggio del LED, il rilevamento di *Out of service* mediante sonar, la web-GUI definitiva e il deployment fisico sul Pico W sono rinviati allo Sprint 2.

# 2 Requirement analysis

L'analisi dei requisiti svolta nello Sprint 0 costituisce la base di questo sprint.
In particolare, si confermano e si riprendono i seguenti punti:

- **Natura del sistema**: si conferma la natura distribuita ed eterogenea del sistema.
- **Modello delle entità**: si riprendono `cargoservice`, `cargorobot`, IOPort, sonar e LED con le responsabilità individuate nello Sprint 0. L'articolazione software delle macro-parti viene approfondita nell'analisi del problema.
- **Componente del robot da riusare**: si conferma l'impiego di `RobotSmart26`, poiché espone un'interfaccia a messaggi orientata al raggiungimento di destinazioni nella mappa della stiva, in linea con i concetti espressi nei requisiti (IOPort, `slot5`, `slot1`–`slot4`, HOME).
- **Validazione delle ipotesi dello Sprint 0**: il committente ha confermato le ipotesi formulate:

  **A1** *Tempi di attesa*: finestra massima di posizionamento $T_a = 30\,\text{s}$ dall'accettazione della richiesta e conferma di presenza per $T_p = 3\,\text{s}$ continui con $D < D_{\text{FREE}}/2$, da completarsi entro la scadenza di $T_a$.

  **A2** *Una sola operazione alla volta*: il servizio gestisce un solo caricamento per volta e risponde `retrylater` a eventuali richieste sopraggiunte mentre è *engaged*.

  **A3** *Ciclo di vita dello slot*: lo slot viene riservato all'accettazione, se scade il timeout $T_a$ la prenotazione viene annullata e lo slot torna libero, mentre a deposito ultimato lo slot risulta definitivamente occupato.

## 2.1 Ulteriori chiarimenti

Sono stati discussi e concordati con il committente i seguenti aspetti:

**Interazione con il marker in `slot5`.**
**Domanda.** Come viene attivata la marcatura, quanto dura e come viene notificato il completamento?

**In accordo col committente**, il marker non sarà un dispositivo fisico, ma la marcatura sarà semplicemente simulata tramite un'attesa di durata di N secondi. L'operazione parte in automatico appena `cargorobot` deposita il container in `slot5`, una volta trascorsi i secondi previsti, `cargoservice` ordina a `cargorobot` di prelevare il container per spostarlo nello slot di destinazione.

**Ritorno alla posizione HOME.**
**Domanda.** Dopo il deposito del container nello slot riservato, il robot deve tornare alla posizione HOME? In quale momento lo stato della stiva viene aggiornato sul display e quando il servizio può accettare una nuova richiesta?

**In accordo col committente**, il robot tornerà alla posizione HOME dopo ogni deposito nello slot finale. Lo slot riservato diventerà effettivamente *occupato* (con conseguente aggiornamento dello stato della stiva sul display) non appena il deposito nello slot è completato, mentre il sistema resterà *engaged* durante il tragitto di rientro del robot. Soltanto dopo la conferma del raggiungimento di HOME il sistema tornerà *disengaged* e potrà accettare una nuova richiesta.

**Gestione del fallimento del robot.**
**Domanda.** Se `cargorobot` non riesce a raggiungere una destinazione (IOPort, `slot5`, slot riservato o HOME), `cargoservice` deve ritentare il movimento, annullare l'operazione oppure impostare il sistema *Out of service*?

**In accordo col committente**, un errore di navigazione porterà il servizio nello stato *Out of service*, impedendo l'accettazione di nuovi carichi. La prenotazione dello slot in corso non verrà liberata automaticamente, poiché dopo un guasto in questo contesto la posizione effettiva del container richiede una verifica manuale.

**Ripristino dalla condizione *Out of service*.**
**Domanda.** Il sistema può uscire automaticamente dallo stato *Out of service* e tornare ad accettare richieste?

**In accordo col committente**, il sistema non effettuerà un ripristino automatico. Una volta entrato nello stato *Out of service*, il servizio manterrà tale condizione, mostrerà il relativo messaggio sulla IOPort e risponderà `retrylater` alle nuove richieste. Il ritorno all'operatività richiederà un intervento di manutenzione esterno al normale ciclo applicativo.

**Conferma di prelievo e deposito del container.**
**Domanda.** Quale componente conferma che il container è stato effettivamente prelevato o depositato e come?

**In accordo col committente**, poiché `RobotSmart26` si occupa unicamente della navigazione nella stiva fino alla destinazione richiesta, le operazioni di presa e rilascio del container si considerano parte del compito svolto da `cargorobot` una volta giunto sul posto. Pertanto, la conferma di completamento fornita da `cargorobot` attesta sia il raggiungimento della posizione sia il completamento dell'operazione di prelievo o deposito.

**Associazione del barcode al container durante la marcatura.**
**Domanda.** Il testo dei requisiti specifica che in `slot5` il marker applica al container un barcode identificativo e segnala il completamento dell'attività: il codice generato deve essere acquisito dal servizio e associato allo slot in cui il container viene depositato?

**In accordo col committente**, al termine della marcatura al container viene associato un barcode. Tale identificativo accompagna il container fino al termine del caricamento e, una volta concluso il deposito finale, rimane associato allo slot occupato (`slot1`–`slot4`) per identificare il carico presente nella stiva.

**Criterio di selezione dello slot nella stiva.**
**Domanda.** In presenza di più slot liberi tra `slot1` e `slot4`, quale criterio deve adottare `cargoservice` per scegliere lo slot da riservare alla richiesta di carico?

**In accordo col committente**, l'inserimento nella stiva segue l'ordine di indice minore: a ogni richiesta accettata viene sempre selezionato e riservato il primo slot disponibile procedendo da `slot1` a `slot4`. Se tutti e quattro gli slot risultano occupati, la richiesta viene rifiutata con `load_refused`.

# 3 Problem Analysis

L'analisi del problema descrive come organizzare il sottosistema dello Sprint 1, esaminando le fasi attraversate da un caricamento, i dati necessari e le comunicazioni tra gli attori, per poi affrontare i dettagli di implementazione nella progettazione.

## 3.1 Fasi del caricamento e modello operativo

In `cargoservice` vengono mantenute separate la **fase del caricamento** e lo **stato del servizio**, affiancate dallo stato della navigazione gestito da `cargorobot`:

```
phase           = DISENGAGED | WAIT_CONTAINER | TO_IOPORT | TO_SLOT5 | WAIT_MARKER | TO_FINAL |
    RETURN_HOME
serviceStatus   = WORKING | OUT_OF_SERVICE
navigationState = IDLE | MOVING | ARRIVED | FAILED
```

Quando non c'è un caricamento attivo la fase è `DISENGAGED`, mentre le altre fasi seguono il lavoro da svolgere: attendere il container, raggiungere la IOPort e prelevarlo, portarlo in `slot5`, attendere la marcatura, trasferirlo nello slot finale e tornare a `HOME`, mantenendo il sistema *engaged* per tutta la sequenza.

`cargorobot` distingue tra attesa di un comando, movimento, arrivo e fallimento, comunicando l'esito a `cargoservice`, che mantiene `serviceStatus` separato dalla fase del carico perché un'anomalia può verificarsi anche a operazione in corso e portare il sistema in `OUT_OF_SERVICE`.

## 3.2 Dalla richiesta al deposito finale

Alla ricezione di una `load_request`, il servizio valuta la disponibilità del sistema: se la IOPort è occupata, il sistema è fuori servizio o è già in corso un caricamento (phase != DISENGAGED), la richiesta viene rinviata con un `retrylater`. Se invece il sistema è operativo ma non vi sono slot liberi, risponde con `load_refused`.

Altrimenti prenota lo slot libero con indice minore e ne restituisce il nome con `load_accepted`, memorizzandolo nell'operazione corrente e facendolo passare da `FREE` a `RESERVED`, quindi entra in `WAIT_CONTAINER` e avvia i 30 s di attesa.

- Se la presenza del container non viene confermata in tempo, la prenotazione viene annullata e lo slot torna `FREE`, mentre il servizio torna `DISENGAGED` senza muovere il robot.
- Se la conferma arriva entro la scadenza e il servizio è operativo, `cargoservice` interrompe il timeout e avvia il trasporto, inviando richieste `reach_target` a `cargorobot`.

Per ogni tratta, `cargoservice` indica la destinazione logica e attende `target_reached` oppure `target_failed`, registrando l'eventuale errore ed entrando in `OUT_OF_SERVICE` senza annullare la prenotazione. Quando il deposito finale è confermato, lo slot passa a `OCCUPIED` e il servizio aggiorna il display, ma per concludere il ciclo e tornare `DISENGAGED` deve ancora attendere il rientro del robot a HOME.

```
DISENGAGED      -- richiesta accettata ---------> WAIT_CONTAINER
WAIT_CONTAINER  -- timeout 30s -----------------> DISENGAGED
WAIT_CONTAINER  -- presenza confermata (3s) ----> TO_IOPORT
TO_IOPORT       -- prelievo confermato ---------> TO_SLOT5
TO_SLOT5        -- deposito temporaneo ok ------> WAIT_MARKER
WAIT_MARKER     -- marcatura completata --------> TO_FINAL
TO_FINAL        -- deposito finale confermato --> RETURN_HOME
RETURN_HOME     -- rientro in HOME confermato --> DISENGAGED
```

## 3.3 Layout e gestione operativa

La gestione dello spazio richiede di distinguere due livelli: la navigazione deve conoscere la disposizione della stiva e gli ostacoli, mentre il coordinamento del carico deve conoscere le destinazioni logiche e la disponibilità delle aree. `cargoservice` può quindi richiedere il raggiungimento di IOPort, `slot5`, slot riservato e HOME senza calcolare direttamente i percorsi, affidati a `RobotSmart26`.

Occorre inoltre distinguere lo stato degli slot di deposito, la presenza temporanea di un container nell'area di marcatura e i dati dell'operazione di carico in corso. Queste informazioni hanno significati diversi: prenotare uno slot non equivale a occuparlo, e la sosta in `slot5` non costituisce ancora un deposito finale.

La conoscenza usata per raggiungere le aree deve essere coerente con la disposizione della stiva. La rappresentazione della mappa, le coordinate e l'organizzazione dei dati in componenti software sono definite nella sezione Project.

## 3.4 Architettura logica

L'architettura logica distingue il coordinamento del carico, il controllo del robot, l'interazione con il cliente e il rilevamento della presenza, affiancati dai dati del dominio e dal servizio esterno di navigazione.

Rispetto al modello iniziale dello Sprint 0, in cui sonar, LED e IOPort erano individuati come entità distinte, l'analisi del problema porta a introdurre due attori dedicati, `sensorservice` e `ioportservice`, accanto a `cargoservice` e `cargorobot`.

### Dal Sonar e LED all'introduzione di `sensorservice`

Secondo i requisiti, il sonar misura la distanza $D$ per rilevare la presenza di un container quando $D < D_{\text{FREE}}/2$ per 3 s, mentre il LED deve lampeggiare finché il sistema è *engaged*. Valutando l'impatto di queste attività su `cargoservice`, emergono tre aspetti principali da gestire:

1. **Differenza tra misura grezza ed evento**: il sonar genera un flusso continuo di letture, ma a `cargoservice` interessa solo sapere quando la distanza resta sotto soglia per 3 s consecutivi. Se il coordinatore dovesse analizzare ogni singolo campione, finirebbe per mescolare il filtraggio del segnale con la logica di business legata a prenotazioni, timeout e spostamenti del robot.
2. **Monitoraggio continuo della porta**: una richiesta di carico deve essere rifiutata con `retrylater` anche quando la IOPort è già occupata. Dato che un container può essere appoggiato in qualsiasi momento (anche prima di premere il pulsante) e servono comunque 3 s per confermarne la presenza, la lettura del sonar non può essere attivata solo dopo aver accettato una richiesta, ma deve restare sempre attiva in modo indipendente dallo stato di `cargoservice`.
3. **Gestione del lampeggio del LED**: il LED deve lampeggiare per tutto il tempo in cui il sistema è *engaged*. Si tratta di un'attività periodica continua, mentre per `cargoservice` l'accensione e lo spegnimento rappresentano comandi puntuali legati all'inizio e alla fine del ciclo di carico.

Per non appesantire il coordinatore, la gestione di sonar e LED è stata affidata a un attore dedicato, `sensorservice`, distinto da `ioportservice` che si occupa invece dell'interazione con l'utente:

- `ioportservice`: raccoglie la pressione del pulsante e aggiorna il display con le risposte del servizio e lo stato della stiva.
- `sensorservice`: filtra le letture del sonar per notificare gli eventi di presenza e controlla il LED in base ai comandi ricevuti.

| Aspetto del problema | Gestione diretta in `cargoservice` | Delega a `sensorservice` |
|---|---|---|
| **Verifica di $D < D_{\mathrm{FREE}}/2$ per 3 s** | Il coordinatore deve valutare ogni campione mentre gestisce richieste, timer e mosse del robot. | Il controllo temporale è isolato in un componente dedicato, più semplice da testare. |
| **Controllo IOPort già occupata** | Obbliga `cargoservice` a elaborare le letture del sonar anche quando è `DISENGAGED`. | Il monitoraggio è sempre attivo: `cargoservice` riceve solo la notifica di presenza ed è subito pronto a rispondere `retrylater`. |
| **Indipendenza dall'hardware** | Il coordinatore dipende da come vengono letti i campioni e pilotato il LED. | `cargoservice` scambia solo messaggi logici, restando invariato sia in simulazione sia con i dispositivi fisici. |

**Componenti architettura logica**

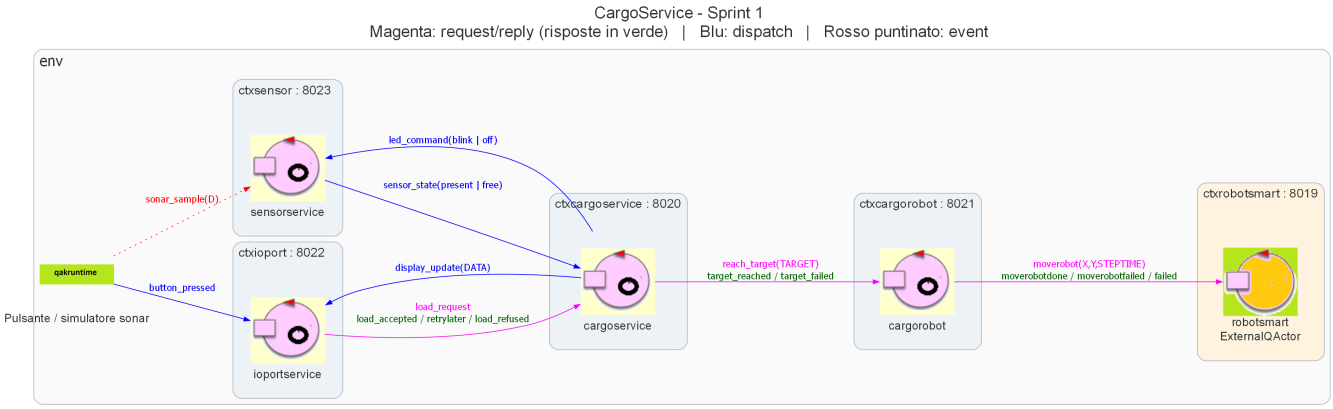
La tabella seguente riassume i componenti individuati nell'analisi del problema e le rispettive responsabilità:

| Componente | Tipo | Responsabilità |
|---|---|---|
| `cargoservice` | Attore QAK | Coordina il carico e gestisce richieste, prenotazioni e attese, essendo l'unico attore che modifica gli oggetti di stato del dominio. |
| `cargorobot` | Attore QAK | Riceve una destinazione logica, richiede la navigazione a `RobotSmart26` e comunica l'esito con `target_reached` o `target_failed`. |
| `ioportservice` | Attore QAK | Inoltra le richieste del pulsante e riceve risposte e aggiornamenti da mostrare sul display. |
| `sensorservice` | Attore QAK | Interpreta le misure del sonar, notifica la presenza confermata del container e gestisce i comandi logici per il LED. |
| Modello del dominio | Dati del carico | Comprende disponibilità degli slot, container nell'area di marcatura e operazione corrente, separati dalle informazioni spaziali necessarie alla navigazione. |
| `RobotSmart26` | Servizio esterno | Pianifica ed esegue i movimenti nell'ambiente virtuale, comunicando l'esito della navigazione a `cargorobot`. |

Solo `cargoservice` aggiorna lo stato degli slot, dell'area di marcatura e dell'operazione corrente, evitando aggiornamenti concorrenti degli stessi dati da parte di componenti diversi e offrendo un unico riferimento per lo stato della stiva.

**Lo stato della stiva.** Lo stato degli slot è gestito internamente da `cargoservice`. Per l'osservabilità dei test viene adottata un'interazione di tipo **pull**: i client consultano lo stato mediante `get_hold_status`, senza dover ricevere notifiche a ogni variazione. Gli aggiornamenti del display vengono invece comunicati alla IOPort attraverso il componente di interfaccia. La

richiesta `setup_hold` permette di predisporre la configurazione iniziale della stiva. Le modifiche sono consentite solo in assenza di caricamenti in corso.




# 4 Test Plans

I piani di test riprendono quelli dello Sprint 0 e coprono l'intero ciclo di carico attraverso test JUnit collegati via TCP ai contesti del sistema.

Essi assumono che sia possibile predisporre lo stato iniziale della stiva e osservare gli effetti delle operazioni.

**T1 – Accettazione della richiesta.**
Si occupa `slot1` con `setup_hold(slot1_occupied)`, lasciando liberi gli altri slot, e si invia una `load_request`, alla quale il servizio deve rispondere `load_accepted(slot2)` dopo aver prenotato `slot2` e averlo portato in `RESERVED`. Il servizio deve inoltre inviare `led_command(blink)` ed entrare in `WAIT_CONTAINER`, mentre il robot resta fermo a `HOME`.

**T2 – Richiesta quando il servizio è già in uso (`retrylater`).**
Si verificano due situazioni separate: la IOPort è già occupata, oppure arriva una nuova richiesta mentre il servizio è *engaged*, in attesa del container o durante il trasporto. In entrambi i casi la risposta attesa è `retrylater`, senza prenotare altri slot né modificare l'operazione in corso.

**T3 – Richiesta rifiutata a stiva piena (`load_refused`).**
Si lascia libera la IOPort e si occupano i quattro slot con `setup_hold(full)`, quindi si invia una `load_request` verificando che il servizio risponda `load_refused` e che `get_hold_status` confermi sia lo stato invariato degli slot sia la permanenza del servizio in `DISENGAGED`.

**T4 – Fuori servizio per fallimento navigazione.**
Durante una tratta si fa rispondere `target_failed` a `cargorobot`, verificando che `cargoservice` passi a `OUT_OF_SERVICE` e invii `display_update(Out_of_service)`, per poi controllare che un'ulteriore `load_request` riceva `retrylater` senza modificare le prenotazioni.

**T5 – Container non arrivato entro il timeout.**
Si parte da una stiva vuota e si accetta una richiesta che porta `slot1` in `RESERVED`, lasciando poi libera la IOPort per 30 s. Alla scadenza il servizio deve liberare lo slot, spegnere il LED, aggiornare il display e tornare `DISENGAGED` senza aver mosso il robot, mentre la lettura con `get_hold_status` deve confermare `slot1=FREE` e la conclusione dell'attesa.

**T6 – Rilevamento e movimento verso `slot5`.**
Dopo l'accettazione si inviano a `sensorservice` campioni con $D = 40 < D_{\mathrm{FREE}}/2$ per almeno 3 s continui, entro i 30 s disponibili, verificando che la notifica a `cargoservice` parta solo quando è stata raggiunta la durata richiesta. A quel punto il servizio deve avviare il prelievo alla IOPort e il trasferimento verso il punto di accesso $A_5$ di `slot5`, mantenendo lo slot finale in `RESERVED`.

**T7 – Ciclo completo di caricamento, marcatura e rientro in HOME.**
Si segue un caricamento completo, dal prelievo del container alla IOPort al deposito in `slot5`, poi, dopo l'attesa di marcatura e la generazione del barcode, fino al trasferimento nello slot prenotato, `slot1`, che deve passare a `OCCUPIED` con un aggiornamento immediato del display e `slot5` libero. Il servizio deve restare impegnato fino al rientro del robot a `HOME`, per poi spegnere il LED e tornare `DISENGAGED`.

# 5 Project

## 5.1 Comportamento degli attori

**Contratti di comunicazione.** Ogni attore è modellato come una macchina a stati finiti di Moore, con azioni associate allo stato corrente. Le comunicazioni tra gli attori si basano su tre tipi di messaggi:

- **Request / Reply**, usato per `load_request`, `reach_target` e `moverobot` (oltre alle interazioni di osservabilità e controllo `get_hold_status` e `setup_hold`) quando il mittente deve attendere l'esito della richiesta prima di proseguire con l'operazione che ne dipende.
- **Dispatch**, usato per `sensor_state`, `led_command` e `display_update` quando occorre notificare uno stato o richiedere un'azione senza prevedere una risposta.
- **Event**, usato per `sonar_sample` dal simulatore, che emette i campioni di distanza senza dover conoscere il componente che li userà.

La tabella segue lo scambio dei messaggi durante il caricamento:

| Fase | Azione e messaggio | Componenti che comunicano |
|---|---|---|
| Richiesta di carico | Invio richiesta (`load_request`) | `ioportservice` → `cargoservice` |
| Accettazione e attesa | Risposta alla richiesta (`load_accepted` / `retrylater` / `load_refused`) | `cargoservice` → `ioportservice` |
| | Attivazione lampeggio LED (`led_command(blink)`) | `cargoservice` → `sensorservice` |
| Rilevamento del container | Acquisizione distanza (`sonar_sample(D)`) | Simulatore → `sensorservice` |
| | Conferma presenza per 3 s (`sensor_state(present)`) | `sensorservice` → `cargoservice` |
| Prelievo e trasporto | Richiesta di spostamento (`reach_target(TARGET)`) | `cargoservice` → `cargorobot` |
| (`IOPORT`, `SLOT5`, `SLOTn`, `HOME`) | Richiesta di navigazione verso $A_n$ (`moverobot(X,Y)`) | `cargorobot` → `robotsmart` |
| | Esito navigazione (`moverobotdone` / `failed`) | `robotsmart` → `cargorobot` |
| | Conferma tappa o fallimento (`target_reached` / `target_failed`) | `cargorobot` → `cargoservice` |
| Deposito e conclusione | Aggiornamento display (`display_update(DATA)`) | `cargoservice` → `ioportservice` |
| | Spegnimento LED (`led_command(off)`) | `cargoservice` → `sensorservice` |
| Osservabilità e test | Impostazione stato stiva (`setup_hold(MODE)` / `preset_ack`) | Client di test ↔ `cargoservice` |
| | Lettura stato stiva e servizio (`get_hold_status` / `hold_state(DATA)`) | Client di test ↔ `cargoservice` |

**`cargoservice`.** L'attore traduce in stati QAK le fasi del caricamento individuate nell'analisi, passando da uno stato all'altro in base ai messaggi ricevuti e alle scadenze temporali.

La ricezione di `sensor_state(present)` da `sensorservice` aggiorna l'informazione di presenza alla IOPort. Se il servizio è in `WAIT_CONTAINER`, è operativo e la conferma arriva entro la scadenza dell'operazione corrente, `cargoservice` disattiva l'attesa e avvia il prelievo tramite `cargorobot`. Se invece non è in attesa di un container, la notifica aggiorna soltanto lo stato della porta e non avvia un trasporto; una nuova richiesta con IOPort occupata riceverà `retrylater`.

Al deposito finale, l'aggiornamento dello slot viene comunicato a `ioportservice` tramite `display_update(DATA)`, mantenendo `cargoservice` responsabile dei dati da mostrare e affidando all'attore di interfaccia la loro presentazione.

Se arriva una nuova `load_request` mentre è già in corso un caricamento, l'attore passa allo stato `busy_retrylater`, risponde `retrylater` e torna allo stato in cui si trovava, senza modificare l'operazione in corso né riavviare i timer.

La ricezione di `target_failed` durante una tratta porta nello stato `robot_failure`: il servizio imposta `OUT_OF_SERVICE`, spegne il LED e invia `display_update(Out_of_service)` alla IOPort, lasciando invariati lo stato della stiva e i dati del carico interrotto.

**`cargorobot`.** L'attore alterna lo stato di attesa `idle` allo stato `moving`. Alla ricezione di `reach_target(TARGET)`, legge da `Layout` le coordinate del punto di accesso $A_n$ e invia `moverobot(X,Y)` a `robotsmart`.

Quando riceve `moverobotdone`, considera conclusa anche l'operazione di presa o rilascio sul posto e risponde `target_reached` a `cargoservice`, se invece riceve un errore dal navigatore, restituisce `target_failed`. In entrambi i casi, inviata la risposta, torna in `idle`.

**`sensorservice`.** Nello stato `monitoring`, l'attore resta in ascolto degli eventi `sonar_sample(D)` e dei comandi `led_command(blink|off)` (il collegamento al LED fisico è previsto nello Sprint 2).

Ogni campione viene valutato nello stato `eval_sample`: finché la distanza resta sotto $D_{\text{FREE}}/2$, l'attore aggiorna la durata del rilevamento e, raggiunti i 3 s continui, invia `sensor_state(present)` a `cargoservice`. Se arriva una misura con $D \geq D_{\text{FREE}}/2$, azzera il conteggio e torna in `monitoring`.

**`ioportservice`.** Alla pressione del pulsante, l'attore passa da `idle` a `requesting` e invia `load_request` a `cargoservice`. Mostrata sul display la risposta ricevuta (`load_accepted`, `retrylater` o `load_refused`), torna in `idle`, dove riceve anche gli aggiornamenti inviati con `display_update(DATA)`.

## 5.2 Scelte di realizzazione

### 5.2.1 Layout e gestione operativa

La gestione dello spazio è divisa su due livelli: il robot (`RobotSmart26`) usa la mappa per l'evitamento ostacoli e il pathfinding con $A^*$, mentre la logica applicativa traccia gli slot, il loro stato e i rispettivi punti di accesso.

L'insieme di queste informazioni viene incapsulato in quattro componenti:

- **`Layout`** contiene la mappa statica della stiva, le coordinate delle destinazioni e le aree degli slot, che `cargorobot` consulta in sola lettura.
- **`HoldInventory`** conserva barcode e stati degli slot `1`–`4`, ossia `FREE`, `RESERVED` e `OCCUPIED`, permettendo a `cargoservice` di assegnare uno slot senza occuparsi delle sue coordinate.
- **`MarkingBuffer`** tiene traccia del container in sosta temporanea in `slot5`, separatamente dagli slot di deposito finale.
- **`LoadOperation`** raccoglie i dati del caricamento in corso: ID, slot assegnato, barcode, fase, scadenza e spostamento attivo.

**Conoscenza dell'ambiente e coerenza delle mappe.** `cargoservice` coordina il ciclo di carico usando i nomi delle destinazioni, mentre il calcolo dei percorsi è affidato a `RobotSmart26`, evitando di conservare nel servizio una copia della mappa degli ostacoli che dovrebbe essere mantenuta coerente con quella del navigatore.

Le coordinate dei punti di accesso $A_n$ definite in `Layout` e la mappa di `RobotSmart26` vengono configurate a partire dalla stessa disposizione della stiva e verificate attraverso i test di integrazione, senza introdurre un protocollo di sincronizzazione durante l'esecuzione.

**Configurazione spaziale.** La stiva viene rappresentata con una griglia di sei righe e otto colonne, nella quale ogni cella è identificata dalla coppia di indici `cells[row][column]`. La mappa comprende `HOME`, la `IOPort`, gli slot di deposito `slot1`–`slot4` e l'area di marcatura `slot5`, con le celle $A_n$ a indicare i punti di accesso preferenziali ai rispettivi slot.

| Destinazione | Cella dell'area | Punto di accesso |
|---|---|---|
| `HOME` | $(0, 0)$ | $(0, 0)$ |
| `IOPort` | $(4, 0)$ | $(4, 0)$ |
| `slot1` | $(1, 2)$ | $A_1 = (0, 2)$ |
| `slot2` | $(1, 3)$ | $A_2 = (0, 3)$ |
| `slot3` | $(3, 2)$ | $A_3 = (4, 2)$ |
| `slot4` | $(3, 3)$ | $A_4 = (4, 3)$ |
| `slot5` | $(2, 4)$ | $A_5 = (2, 5)$ |

### 5.2.2 Rappresentazione del dominio

I concetti del dominio, come gli slot e le destinazioni del robot, vengono rappresentati con oggetti dedicati (POJO), così da raccogliere in un unico modello le informazioni che li descrivono e le operazioni necessarie per gestirli. Prima di scrivere le classi concrete, si definisce quali operazioni questi oggetti devono offrire attraverso le interfacce Java, mentre le enumerazioni descrivono i valori ammessi, come gli stati di uno slot.

```
enum SlotState { FREE, RESERVED, OCCUPIED }
enum Target { HOME, IOPORT, SLOT1, SLOT2, SLOT3, SLOT4, SLOT5 }

interface Position {
  int row();
  int column();
}

interface Destination {
  Set<Position> area();   // insieme celle che costituiscono la destinazione
  Position accessPoint(); // coordinate per casella di accesso An
}

interface Layout {
  int rows();
  int columns();
  Destination destination(Target target); // traduce target logico in area e punto di accesso An
}

interface HoldInventory { // slot 1-4
  SlotState stateOf(int slotId);
  OptionalInt reserveFirstAvailable();                 // prenota primo slot disponibile
  void releaseReservation(int slotId);                 // rilascia prenotazione per timeout
  void confirmDeposit(int slotId, String containerId); // occupa slot e lo associa al barcode
}

interface MarkingBuffer { // slot5
  Optional<String> containerId();          // identificatore container presente
  void confirmDeposit(String containerId); // conferma deposito in slot5
  String confirmCollection();              // svuota buffer e restituisce il barcode
}
```

**Gestione delle attese.** Il timeout di 30 s per il posizionamento del container viene avviato una sola volta all'accettazione della richiesta e, alla scadenza, genera il messaggio `wait_timeout`. In questo modo il conteggio non riparte da capo se l'attore esce momentaneamente dallo stato di attesa per rispondere `retrylater` a un'altra richiesta.

Se `sensor_state(present)` arriva in tempo, il timeout viene disattivato e un eventuale `wait_timeout` già in coda viene ignorato negli stati successivi. Per la marcatura in `slot5` basta

invece un'attesa configurabile di $N$ secondi prima di associare il barcode e far ripartire il robot.

**Simulazione del sonar.** In assenza del sensore fisico, gli eventi `sonar_sample(D)` vengono generati da una sequenza configurabile oppure inviati direttamente dai test JUnit, così da provare in modo ripetibile sia i 3 s continui sia l'interruzione del conteggio.

**Contesti di esecuzione.** Nel prototipo ogni attore viene assegnato a un contesto QAK separato. Questa suddivisione permette di avviare i componenti in modo indipendente, mantenendo la comunicazione attraverso i messaggi QAK e lasciando separata la scelta dei nodi fisici su cui eseguire i contesti. I contesti sono:

- `ctxcargoservice`, port `8020`: racchiude la logica di business e la gestione dello stato della stiva, mantenendo il coordinamento centrale separato e protetto da eventuali errori dei moduli operativi.
- `ctxcargorobot`, port `8021`: si occupa di convertire i target logici nelle coordinate della griglia e di richiedere gli spostamenti a `RobotSmart26`. La separazione permette di gestire la navigazione come un servizio autonomo, pronto per essere spostato a bordo del robot.
- `ctxioport`, port `8022`: gestisce i componenti della banchina (pulsante e display), disaccoppiando l'I/O dal servizio centrale e predisponendo il collegamento con la futura interfaccia web.
- `ctxsensor`, port `8023`: elabora le letture del sonar per verificare la presenza del container e riceve i comandi per il LED. Nello Sprint 2 comunicherà tramite MQTT con il Raspberry Pi Pico W, sul quale saranno realizzati l'acquisizione delle distanze e il controllo fisico del LED.

`RobotSmart26` gira autonomamente sulla port `8019` come servizio esterno (`ExternalQActor`).

### 5.3 Preparazione della dimostrazione

La dimostrazione usa `RobotSmart26` collegato a WEnv. All'avvio, la scena e il navigatore devono essere entrambi in `HOME`, `cargoservice` deve risultare `WORKING` e `DISENGAGED`, con tutti gli slot liberi.

La richiesta `load_request` prenota `slot1`, ma il robot parte soltanto dopo che il sonar ha confermato la presenza del container per almeno 3 s continui, entro i 30 s dall'accettazione. Nella dimostrazione i campioni sono prodotti da un simulatore.

Il percorso atteso è `HOME` → `IOPort` → `slot5` → `slot1` → `HOME`. In `slot5` il robot attende circa 5 s per la marcatura simulata. Al termine, `slot1` deve risultare `OCCUPIED` con barcode, `slot5` libero e il servizio nuovamente `DISENGAGED`.

## 6 Deployment

### 6.1 Avvio del prototipo e dei test

Per eseguire il prototipo distribuito e i test funzionali si procede così:

1. **Avviare l'ambiente virtuale e il broker MQTT**: dalla cartella che contiene i file Docker forniti dal committente, eseguire `docker compose -f mosquitto.yml up -d` e `docker compose -f vrWithGui26.yaml up -d`, quindi aprire `http://localhost:8090`.
2. **Avviare `RobotSmart26`**: entrare nella cartella `robotsmart26` ed eseguire `./gradlew run` per avviare il servizio sulla port `8019`.

3. **Avviare i contesti**: avviare i quattro contesti distribuiti del prototipo:
   - `./gradlew runCtxCargoService`
   - `./gradlew runCtxCargoRobot`
   - `./gradlew runCtxIoPort`
   - `./gradlew runCtxSensor`
4. **Eseguire i test JUnit**: lanciare `./gradlew test` e consultare il report generato in `build/reports/tests/test/index.html`.

# 7 Goal dello Sprint 2

Lo Sprint 2 riprende le parti rimaste da completare:

1. gestire il passaggio a *Out of service* quando il sonar misura $D > D_{\text{FREE}}$ per almeno 3 s continui, mostrando il messaggio sul display.
2. realizzare la IOPort come web-GUI, con lo stato della stiva e i messaggi del servizio aggiornati in tempo reale.
3. integrare sonar e LED sul Pico W.